

Taller Práctico / Parcial: 5 Ejercicios de Query Methods (Guía de Trabajo)

Asignatura: Computación en Internet II (Compunet II) -- Universidad ICESI

Tema: Spring Data JPA - Derived Query Methods (Sin soluciones directas, con pistas guiadas)

Instrucciones:

1. Lee atentamente el contexto de cada problema y el camino de navegación entre entidades.
2. Utiliza las rutas de navegación, las reglas del parser y la tabla general de palabras clave para deducir el nombre del Query Method correspondiente.
3. Escribe la firma en la interfaz del **Repositorio** y su invocación en el **Service** dentro de los bloques de código proporcionados.

Plantilla Anatómica General de un Query Method

Recuerda que Spring Data JPA descompone el nombre del método en 4 bloques léxicos secuenciales:

<u>find / exists / count</u>	<u>[Modificador de Unicidad / Límite]</u>	<u>By [Criterio 1] ... [And / Or] ... [Criterio N]</u>	<u>[OrderBy ... Asc/Desc]</u>
1. Acción	2. Modificadores (Opcional)	3. Criterios de Filtrado	4. Cláusula de Orden (Opcional)

Ejercicio 1: Reporte Docente por Créditos Dictados y Departamento

Contexto del Problema

La decanatura necesita listar a los profesores **activos** (`active = true`) que pertenezcan a un **departamento** específico (sin distinguir mayúsculas/minúsculas) y que se encuentren dictando asignaturas de alta exigencia académica (cursos con una cantidad de **créditos mayor o igual** a un valor mínimo).

Para la entrega del reporte:

- Cada profesor debe aparecer **únicamente una vez** (sin duplicados causados por dictar múltiples cursos).
- La lista debe ordenarse alfabéticamente de forma ascendente por el **apellido** del profesor y, en caso de empate, por su **nombre**.

Pistas y Ayudas Conceptuales

[!TIP] **Ruta de Navegación:** Empiezas en la entidad **Profesor** → navegas a su colección **cursos** → filtras por el campo **creditos** del curso.

- **Repositorio Objetivo:** `ProfesorRepository` (entidad base: `Profesor`).
- **Tipo de Retorno:** `List<Profesor>`
- **Parámetros esperados en la firma:** (`String departamento, int minCreditos`).
- **Aspectos a considerar en el diseño:**
 - ¿Cómo evitas duplicar entidades cuando navegas una colección `OneToMany`?
 - ¿Cómo evalúas un atributo booleano sin obligar a pasar un argumento adicional en la firma?
 - ¿Cómo comparas valores numéricos con la relación de orden mayor o igual?
 - ¿Cómo configuras la insensibilidad a mayúsculas y el ordenamiento compuesto por dos atributos?

Tu Solución:

```
// =====  
// 1. Repositorio: ProfesorRepository.java  
// =====  
  
  
  
  
  
// =====  
// 2. Service: ProfesorService.java  
// =====
```

Ejercicio 2: Búsqueda de Estudiantes por Especialidad Docente y Dominio Institucional

Contexto del Problema

La dirección del programa requiere auditar qué estudiantes **activos** (`active = true`) cuyo **correo institucional termine** con un sufijo o dominio dado (ej. "`@icesi.edu.co`", sin importar mayúsculas/minúsculas), están matriculados en cursos dictados por docentes de una **especialidad** técnica en específico (ej. "`Inteligencia Artificial`", sin distinguir mayúsculas).

El resultado debe listarse sin duplicados y ordenado alfabéticamente por el **apellido** y luego por el **nombre** del estudiante.

Pistas y Ayudas Conceptuales

[!TIP] **Ruta de Navegación (3 niveles de profundidad):**

Estudiante $\xrightarrow{\text{estudianteCursos}}$ EstudianteCurso $\xrightarrow{\text{curso}}$ Curso $\xrightarrow{\text{profesor}}$ Profesor $\xrightarrow{\text{especialidad}}$ String

- **Repositorio Objetivo:** `EstudianteRepository` (entidad base: `Estudiante`).
- **Tipo de Retorno:** `List<Estudiante>`
- **Parámetros esperados en la firma:** (`String dominioCorreo`, `String especialidad`).
- **Aspectos a considerar en el diseño:**
 - ¿Cómo navegas a través de una tabla asociativa intermedia pasando por 3 entidades intermedias?
 - ¿Qué símbolo o convención de nomenclatura de Spring Data debes usar para desambiguar cada nivel de la relación?
 - ¿Qué operador evalúa si una cadena termina con un sufijo dado sin distinguir mayúsculas?

Tu Solución:

```
// =====  
// 1. Repositorio: EstudianteRepository.java  
// =====  
  
// =====  
// 2. Service: EstudianteService.java  
// =====
```

Ejercicio 3: Validación y Métricas sobre la Entidad Intermedia (`EstudianteCurso`)

Contexto del Problema

Durante el proceso de matrícula, se requiere implementar dos consultas derivadas directamente sobre la entidad asociativa intermedia

`EstudianteCurso`:

- **Parte A (Validación booleana):** Verificar si **existe** al menos un estudiante **activo** inscrito con un **profesor en específico** (`profesor.id`) en cursos pertenecientes a un **departamento** determinado (sin distinguir mayúsculas).
- **Parte B (Métricas cuantitativas):** **Contar** el número total de matrículas en materias cuyo **nombre contenga** un fragmento de texto dado (sin distinguir mayúsculas) y cuyos **créditos se encuentren en un rango** inclusivo [`min`, `max`].

Pistas y Ayudas Conceptuales

[!TIP] **Enfoque sobre Entidad Intermedia:** En este ejercicio consultas directamente sobre `EstudianteCurso`, la cual tiene dos relaciones ManyToOne directas: `estudiante` y `curso` (y a su vez `curso` tiene a `profesor`).

- **Repositorio Objetivo:** `EstudianteCursoRepository` (entidad base: `EstudianteCurso`).
- **Parte A:**
 - Tipo de Retorno: `boolean` (operación optimizada de existencia).
 - Parámetros esperados: (`String departamentoCurso`, `Long profesorId`).
 - Aspectos clave: Navegación cruzada bilateral hacia el estudiante activo por un lado, y hacia el departamento y el profesor del curso por el otro.

- **Parte B:**

- Tipo de Retorno: **long** (operación de agregación en el motor SQL).
- Parámetros esperados: (**String subcadenaNombre**, **int minCreditos**, **int maxCreditos**).
- Aspectos clave: Coincidencia parcial de texto en el curso y evaluación de rangos numéricos continuos.

Tu Solución:

```
// =====
// 1. Repositorio: EstudianteCursoRepository.java
// =====
// Parte A (Validación booleana):

// Parte B (Conteo de métricas):

// =====
// 2. Service: MatriculaService.java
// =====
// Invocación Parte A:

// Invocación Parte B:
```

Ejercicio 4: Limitación **Top 5**, Colecciones **In** y Navegación Inversa

Contexto del Problema

Para un informe de decanatura, se requiere consultar los **primeros 5 cursos con mayor cantidad de créditos** que:

1. Pertenezcan a una **lista de departamentos** académicos (ej. ["Ingeniería", "Ciencias Básicas"]).
2. Sean dictados por un profesor con un **apellido** específico (sin distinguir mayúsculas).
3. Tengan matriculado a por lo menos un alumno perteneciente a una **lista de IDs de estudiantes**.

El listado debe estar ordenado de mayor a menor por la cantidad de **créditos** del curso.

Pistas y Ayudas Conceptuales

[!TIP] **Ruta de Navegación Inversa:** Empiezas en **Curso** → navegas a su colección de matrículas **estudianteCursos** → navegas a la entidad **estudiante** → filtras por su identificador **id**.

- **Repositorio Objetivo:** **CursoRepository** (entidad base: **Curso**).
- **Tipo de Retorno:** **List<Curso>**
- **Parámetros esperados en la firma:** (**Collection<String> departamentos**, **String apellidoProfesor**, **Collection<Long> estudiantesIds**).
- **Aspectos a considerar en el diseño:**
 - ¿Qué palabra reservada limita los resultados a nivel de consulta en Spring Data?
 - ¿Cómo se evalúa la pertenencia de un atributo dentro de una colección dinámica de elementos (**Collection<T>**)?
 - ¿Cómo se ordena de forma descendente por una propiedad numérica?

Tu Solución:

```
// =====
// 1. Repositorio: CursoRepository.java
```

```
// =====

// =====
// 2. Service: CursoService.java
// =====
```

Ejercicio 5: Navegación Profunda **ManyToMany** en Cadena (**Usuario** → **Rol** → **Permiso**)

Contexto del Problema

El área de seguridad informática requiere consultar todos los **usuarios activos** (`active = true`) que cumplan simultáneamente dos condiciones de seguridad:

1. Tengan asignado al menos uno de los roles incluidos en una **lista de nombres de roles** permitidos (ej. `["ADMINISTRADOR", "COORDINADOR"]`).
2. Tengan asignado un **permiso con un nombre en específico** (ej. `"GESTION_CURSOS"`, sin distinguir mayúsculas/minúsculas), navegando a través de los permisos asociados al rol.

La consulta debe retornar los usuarios **sin duplicados** y ordenados alfabéticamente por su **apellido** y luego por su **nombre**.

Pistas y Ayudas Conceptuales

[!TIP] **Cadena de Relaciones N:M:**

Usuario $\xrightarrow{\text{roles (List<Rol>)}}$ Rol $\xrightarrow{\text{permisos (List<Permiso>)}}$ Permiso

- **Repositorio Objetivo:** `UsuarioRepository` (entidad base: `Usuario`).
- **Tipo de Retorno:** `List<Usuario>`
- **Parámetros esperados en la firma:** (`Collection<String> nombresRoles, String nombrePermiso`).
- **Aspectos a considerar en el diseño:**
 - ¿Cómo encadenas dos relaciones **ManyToMany** consecutivas en el nombre del método?
 - ¿Cómo filtras el primer nivel con pertenencia a lista y el segundo nivel con coincidencia insensible a mayúsculas?
 - ¿Qué modificador es imprescindible al consultar a través de relaciones de tipo colección para evitar instancias duplicadas?

Tu Solución:

```
// =====
// 1. Repositorio: UsuarioRepository.java
// =====

// =====
// 2. Service: UsuarioService.java
// =====
```

Guía de Referencia General de Palabras Clave de Spring Data JPA

Utiliza esta tabla de apoyo general como referencia de los operadores y modificadores estándar disponibles en el framework:

Categoría	Palabras Clave Soportadas	Propósito General
Acción / Introdutor	<code>findBy...</code> , <code>readBy...</code> , <code>getBy...</code> , <code>queryBy...</code>	Consulta estándar que retorna entidades o listas.

Categoría	Palabras Clave Soportadas	Propósito General
Existencia	<code>existsBy...</code>	Consulta booleana que verifica la presencia de al menos un registro (<code>LIMIT 1</code>).
Conteo	<code>countBy...</code>	Realiza agregación <code>COUNT(*)</code> a nivel de base de datos.
Modificador de Unicidad	<code>Distinct</code>	Elimina filas duplicadas generadas al hacer <code>JOIN</code> con colecciones.
Limitación de Resultados	<code>Top<N></code> , <code>First<N></code>	Limita el conjunto de resultados a <i>N</i> elementos en la cláusula SQL.
Operadores de Comparación	<code>GreaterThan</code> , <code>GreaterThanOrEqual</code> , <code>LessThan</code> , <code>LessThanOrEqual</code> , <code>Between</code>	Comparaciones numéricas, de fechas o de orden.
Operadores de Colección	<code>In</code> , <code>NotIn</code>	Verifica la pertenencia del atributo dentro de una <code>Collection<T></code> .
Operadores de Cadenas	<code>StartingWith</code> , <code>EndingWith</code> , <code>Containing</code> , <code>Like</code>	Coincidencias de prefijo, sufijo o subcadenas.
Modificador de Texto	<code>IgnoreCase</code> , <code>AllIgnoreCase</code>	Ejecuta comparaciones ignorando mayúsculas y minúsculas.
Operadores Booleanos	<code>True</code> , <code>False</code>	Evalúa atributos booleanos sin requerir parámetro en el método.
Nexos Lógicos	<code>And</code> , <code>Or</code>	Combina múltiples condiciones de filtrado.
Ordenamiento	<code>OrderBy...Asc</code> , <code>OrderBy...Desc</code>	Aplica cláusula <code>ORDER BY</code> simple o compuesta.
Desambiguación	<code>_</code> (Guion bajo)	Delimita explícitamente los saltos entre propiedades de entidades anidadas.